



CS361L – Computer Architecture & Parallel Programming Lab

Project Proposal – Fall 2025

Parallel Performance Engineering Using OpenMP – A Practical Exploration of Threading, Scalability, and Execution Behavior.

**2023428 Muhammad Haris
2023559 Mumtaz Ali**

1. Project Overview

The standard OpenMP interface hides a lot of complexity: thread creation, task distribution, load balancing, synchronization, and reduction all happen behind a few simple pragmas. Our project takes a slightly different route. Instead of relying on OpenMP directly, we plan to **recreate the core behavior of OpenMP directives ourselves using low-level C programming and POSIX threads**.

The intention behind this approach is to understand *how* OpenMP actually behaves beneath the surface. By building our own versions of `parallel`, `for`, and scheduling mechanisms, we expect to gain a more realistic picture of the overheads and decisions that influence performance on shared-memory systems.

This project is relevant because it connects the two main objectives of the course: **processor behavior** and **parallel execution models**. Instead of treating OpenMP as a black box, we aim to reconstruct its logic step by step, and then analyze how our custom implementation matches or differs from real OpenMP. The final outcome will not only include performance numbers, but also insights into the design trade-offs of multi-threading frameworks.

2. Methodology

Our methodology is divided into three concrete stages:

A. Design & Implementation of Low-Level OpenMP-Style Mechanisms

We will manually implement the core components that OpenMP typically provides:

1. **Thread Pool Construction**
 - Creating and managing a pool of worker threads using `pthread_create()`.
 - Implementing a safe shutdown and join mechanism.
2. **Task / Loop Scheduling**
 - Implementing our own **static scheduling**, where iterations are divided evenly across threads.
 - Implementing a **dynamic scheduling** model, where threads take chunks from a shared task queue.
 - Optional: experimentation with guided scheduling if time allows.
3. **Synchronization Tools**
 - Creating our own lightweight mutex-based locks.
 - Implementing reduction-style operations.
 - Handling shared data safely to avoid race conditions.
4. **Parallel Region Emulation**
 - Designing a function wrapper that behaves like `#pragma omp parallel`, allowing workers to pick up tasks from the scheduling framework.

In short, instead of writing:

```
#pragma omp parallel for schedule(dynamic)
for (i = 0; i < N; i++) { ... }
```

we will write the **entire machinery** that makes this pragma work.

B. Real-World Workload Implementation

To test our custom OpenMP-like engine, we will choose a computational workload such as:

- Matrix multiplication
- Image operation (e.g., blur, grayscale filter)

The workload will be written twice:

1. **Serial version in pure C**
2. **Parallel version running on our self-made thread scheduler**

We may later compare it with actual OpenMP for validation, but this is optional.

C. Performance Measurement & Analysis

We will measure:

- Execution time (serial vs custom-parallel)
- Speedup and efficiency
- Thread utilization and scheduling fairness
- Overhead introduced by our implementation
- Behavior at varying thread counts
- Scaling with workload size

We plan to generate clear graphs and interpret them through the lens of cache behavior, synchronization cost, load balancing, and scheduling design. The analysis will be reasoning-based rather than just numerical.

3. Tools & Resources

- **C language** (GCC compiler)
- **POSIX threads (pthreads)**
- Linux environment (lab machines)

4. Expected Outcomes

By the end of the project, we aim to deliver:

1. **A working custom parallel execution framework** written fully in C using threads and synchronization primitives.
2. **A computational workload** executed through our framework to demonstrate real performance differences.
3. **A detailed performance report** examining:
 - How our custom implementation compares against theoretical OpenMP behavior
 - Scheduling impact
 - Synchronization costs
 - Hardware-level reasoning behind observed results
4. **A transparent explanation** of design decisions and trade-offs, showing what is easy, what is expensive, and what OpenMP hides behind its pragmas.